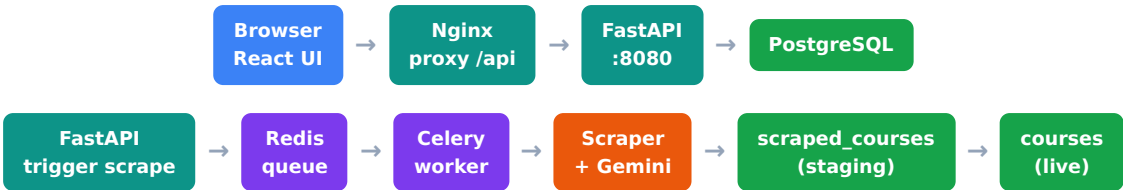


University Portal — System Architecture

Course / Fee / Intake & Requirement Management System | pnpm monorepo | FastAPI + React + Celery + PostgreSQL

A centralised admin portal for universities to manage course data — courses, fees, intakes, scholarships and admission requirements — with AI-powered web scraping, bulk Excel upload/download, and change detection. The system is a **pnpm monorepo**: a React/Vite frontend talks to a Python FastAPI backend (port 8080). Heavy scraping work runs asynchronously in Celery workers (Redis broker), with Gemini AI as a fallback extractor. All data is persisted in PostgreSQL.

End-to-end request & data flow



Layer 1 — Frontend

Browser / Admin Portal `artifacts/university-portal/`

Framework	React 18 + Vite, TypeScript 5.9
UI	shadcn/ui (Radix primitives) + Tailwind CSS
Data fetching	TanStack React Query — server-state caching, mutations, auto-refetch
Routing	Wouter — lightweight client-side SPA routing
Auth	HTTPOnly JWT cookie session (7-day). AuthGuard wraps all protected routes; <code>src/context/auth.tsx</code> exposes <code>useAuth()</code> + <code>AuthProvider</code>
Pages	Dashboard · Universities · Courses · Scraper Jobs · Import History · Scraped Changes
API comms	Relative URLs through the shared Nginx proxy; Orval-generated React Query hooks from the OpenAPI spec

How it works: On load, the app calls `GET /api/auth/me`. No valid session → redirect to `/login`. Each page uses React Query hooks to fetch from the API; mutations (create/edit/approve) invalidate the relevant query keys so the UI refreshes automatically.

Layer 2 — API Server (FastAPI)

FastAPI :8080 (Uvicorn)

- Python 3.12, ASGI via Uvicorn
- Dev: port 8080 (`--reload`)
- Prod: port 8000 behind Nginx
- Nginx: `/api` → `127.0.0.1:8080`
- Entry: `backend-py/app/main.py`
- DB: `asyncpg` + `SQLAlchemy` (`async`)

Auth (JWT Cookie)

- `POST /api/auth/login` — sets cookie
- `POST /api/auth/logout` — clears cookie
- `GET /api/auth/me` — session check
- Cookie `session` (HTTPOnly, 7-day)
- Default `admin@university-portal.local`
- Override: `ADMIN_EMAIL` / `ADMIN_PASSWORD`

API Routers `backend-py/app/routers/`

<code>courses.py</code>	Course CRUD + bulk CSV export (with <code>asyncpg</code> <code>NUMERIC</code> →float decimal fix)
<code>universities.py</code>	University CRUD + associated course listing
<code>scraper_jobs.py</code>	Trigger scrape, poll job status, approve/reject scraped changes
<code>bulk_import.py</code>	Excel (.xlsx) upload → validation → <code>import_jobs</code> table
<code>dashboard.py</code>	Stats overview, courses by degree level, upcoming intakes, recent changes
<code>change_history.py</code>	Field-level diff log (before/after values) for the audit trail
<code>auth.py</code>	login / logout / me endpoints

Layer 3 — Background Workers

Celery Worker `app/tasks/`

- 4 concurrent workers, queue `-Q scrape`
- Broker + backend: Redis :6379
- Tasks: `run_scrape()`, `run_repair()`, `bulk_approve()`
- Start: `celery -A app.tasks.celery_app worker --concurrency=4 --beat`

Redis :6379 + Celery Beat

- Redis: in-memory broker only, no persistence (`--save ''`)
- Bind `127.0.0.1:6379`
- Beat: `nightly_sweep` @ 02:00 UTC
- → baseline snapshot → regression diff → drift alert

How background scraping works: The API enqueues a scrape job onto Redis. A Celery worker claims it, runs the full discovery + extraction pipeline, and writes results into the `scraped_courses` staging table. The nightly Beat task snapshots every university, diffs against the previous night, and fires a drift alert if unexpected field-value changes appear.

Layer 4 — Scraper Engine

Orchestrator — 7-tier discovery `orchestrator.py`

Tier 1	BFS crawl — breadth-first link crawl from the base URL
Tier 2	Sitemap — parse <code>sitemap.xml</code> / <code>sitemap_index.xml</code>
Tier 3	Alt-path probes — <code>/courses</code> , <code>/programs</code> , <code>/study</code> , <code>/international</code> ...
Tier 4	Subdomain fallback — <code>handbook.<domain></code> , <code>study.<domain></code> (YAML-configured)
Tier 5	Playwright browser — headless Chrome for JS-heavy sites (RMIT, Deakin, UNSW)
Tier 6	Wayback Machine — archive.org fallback for blocked/broken sites
Tier 7	Discovery-failure alert — fires if <3 course links found after all tiers

Per-university YAML config is loaded at run start into a contextvar and deep-merged with global defaults.

Per-course Pipeline

- `stage_course.py` — write to `scraped_courses`
- Auto-publish gate — **85% completeness floor**
- 13 review fields: name, degree_level, category, mode, location, duration, intakes, fee, description, academic level/score, english_test, other_req
- `approve_course.py` — promote to `courses`
- `bulk_approve.py` — batch promotion
- Repair scrape — back-fill blank fields only (never overwrites)

Gemini AI — `gemini-2.5-flash-lite`

- AI fallback for hard-to-extract fields
- **Skip gate:** skip if ≥90% fields filled at ≥0.70 confidence (saves 30-50%)
- **Circuit breaker:** trips on 5 quota errors / 60s, stays open 5 min
- **Cost ceiling:** per-job USD budget cap
- **Call log:** `gemini_call_log` — tokens, cost, duration, success

Layer 5 — Field Extractors `app/services/scrape/extractors`

Extractor	What it does
<code>course_name.py</code>	DOM title extraction, strips title suffixes / institution tails (driven by YAML <code>strip_title_suffixes</code>)
<code>duration.py</code>	DOM structural pass + sentence tournament; YAML <code>reject_sentence_patterns</code> filters bad text like "up to N years"
<code>fee.py</code>	International tuition extraction, PDF fee-table matching, CRICOS-matched rows, per-course PDF parsing
<code>english_test.py</code>	IELTS / PTE / TOEFL / CAE / DET band scores (overall + sub-bands); per-host PTE blocklist suppresses false positives
<code>location.py</code>	Campus city extraction, country-suffix append, online/virtual strip, slash-separated city normalisation; YAML <code>strip_patterns</code>
<code>intake.py</code>	Intake-month extraction; session→month map (Autumn→Mar, Spring→Jul, Summer→Nov for AU unis)
<code>cricos_code.py</code>	CRICOS code regex, PDF pipeline integration, AU universities only; provenance into <code>scraped_field_evidence</code>
<code>degree_level.py</code>	Degree-level + category classification
<code>description.py</code> / <code>eligibility.py</code>	Course description & eligibility extraction
<code>gemini_primary.py</code> / <code>ai_fallback.py</code>	AI-based extraction when rule-based extractors miss

Layer 6 — Per-University YAML Config System

scraper_config/ backend-py/scraper_config/

defaults.yaml	Conservative global defaults (changing needs a full regression sweep + human approval)
unis/<slug>.yaml	Per-university overrides — 30+ stubs (uwa, acap, bond, csu, monash ...)
config/schema.py	Pydantic UniConfig split into discovery + extraction sections
config/loader.py	Deep-merge chain: defaults → DB scrape_config → per-university YAML
config/context.py	ContextVar[UniConfig] scoped to each scrape job

Architecture rule: ALL per-university quirks live in scraper_config/unis/<slug>.yaml — never in global extractor code. Key fields: strip_title_suffixes, reject_sentence_patterns, strip_patterns, domestic_only, fallback_subdomains, always_sitemap_supplement. This means onboarding a new university needs zero global code changes.

Layer 7 — Database (PostgreSQL)

Core data tables

universities	name, website, country, scrape_config (JSONB)
courses	live courses: name, degree_level, fee, duration, location, mode
intakes	intake months per course
fees	fee breakdown per course
english_requirements	IELTS/PTE/TOEFL/CAE/DET bands
academic_requirements	GPA, score, level
scholarships	scholarship info per course

Scraping & ops tables

scraped_courses	staging: pending→review→approved→promoted
scraped_field_evidence	per-field provenance: method, source, confidence
scrape_runtime_jobs	job stats: found, staged, cost, elapsed
scrape_run_alerts	rule-based alerts per run
discovery_failure_alerts	Tier-7 fired alerts
gemini_call_log	every AI call: tokens, cost, success
import_jobs	Excel import history
change_history	field-level before/after audit log

SQL cost / coverage views

- `v_gemini_cost_by_university` — per-uni Gemini spend
- `v_gemini_cost_by_call_type` — spend by call type
- `v_gemini_top_spenders_30d` — top spenders last 30 days
- `v_gemini_skip_efficiency` — how often the skip gate fires
- `v_cricos_coverage_au` — CRICOS match rate per AU university

Deployment — Production

DigitalOcean Droplet · 159.65.152.72 · Ubuntu 24.04

Git repo	/root/University-and-Course-data
GitHub	github.com/Studyinfocentre/University-and-course-managment
Deploy	git pull origin main + systemctl restart uni-api-py.service uni-celery.service
FastAPI	uni-api-py.service (Uvicorn :8000), Nginx proxies /api → 127.0.0.1:8000
Celery	uni-celery.service (4 workers + beat)
Database	Local PostgreSQL, DB <code>university_portal</code> , owner <code>uniportal</code>

Critical: The DB URL must use `127.0.0.1`, NOT `localhost` — asynpg's SSL hostname verification fails on this server's DNS. Apply all schema changes via direct `psql`; do NOT run alembic on prod (same DNS issue over TCP).

Monorepo Structure (pnpm workspaces)

Directory layout

- `artifacts/university-portal/` — React+Vite UI
- `src/pages, src/components, src/context`
- `backend-py/` — Python backend
 - `app/main.py` — FastAPI entry
 - `app/routers/` — API handlers
 - `app/services/scrapper/` — engine
 - `extractors/` — 15+ extractors
 - `config/` — YAML config pkg
 - `pipelines/` — PDF pipeline
- `scrapper_config/unis/` — per-uni YAMLs
- `scripts/` — ops scripts
- `tests/` — pytest suite

Key ops scripts `backend-py/scripts/`

- `bulk_approve.py` — batch promote staged courses
- `capture_baseline.py` — snapshot all unis
- `regression_sweep.py` — diff snapshots
- `bond_enrich.py` — Bond JSON API enrichment
- `csu_promotion_diagnostic.py`
- `week4_preflight.sh / week5_preflight.sh`
- `cricos_coverage_diagnostic.py`
- `check_gemini_model.py` — verify cost-optimal model
- `apply_migration_*.py` — schema migrations